

Simulação paralela da propagação direcional de incêndio com zonas de contenção

1^o Trabalho Prático (TB1)

Alec Campos Aoki, 15436800
Henrique Vieira Lima, 15459372
João Gabriel Pieroli da Silva, 15678578
Pedro Augusto Ferraro Paffaro, 15483380
Pedro Henrique Perez Dias, 15484075

São Carlos, 22 de setembro de 2026

Resumo

Este trabalho apresenta duas implementações de um autômato celular que simula a propagação de incêndio em uma área florestal sujeita a vento direcional constante e a zonas de contenção ativadas em passos pré-determinados: uma versão sequencial em C (`fire_seq.c`) e uma versão paralela em C/OpenMP (`fire_omp.c`). A estratégia de paralelização explora o paralelismo de dados dentro de cada passo (uma vez que o próximo passo depende diretamente do estado do passo atual), por meio de uma região paralela persistente, laços `omp for` e reduções inteiras sobre os contadores. As duas versões produzem saídas idênticas em todos os campos (exceto o tempo) para as três cargas de teste avaliadas, e o resultado é comprovadamente independente do número de *threads*. Na carga maior (2500×2500 células, 100 passos), o tempo do núcleo caiu de 6,18 para 1,88 segundos com 8 *threads*, um ganho bruto de $3,29\times$, valor próximo do observado nas outras duas cargas ($3,29$ a $3,54\times$). Esse ganho bruto já é, em grande medida, o próprio ganho atribuível à paralelização: isolando a execução da versão paralela com $T = 1$, medimos um *fator monothread* (a razão entre os tempos seriais das duas implementações) de apenas $0,94$ a $0,97\times$ entre as duas implementações, ou seja, praticamente 1, sem vantagem serial relevante de uma versão sobre a outra. O ganho atribuível à paralelização em si é de $S_{\text{par}} \approx 3,5$ a $3,6\times$ com 8 *threads* nominais, o que, normalizado pelos 4 núcleos físicos do processador usado nos experimentos, corresponde a uma eficiência de 88–91%, próxima do linear; normalizada pelas 8 *threads* nominais (metade delas lógicas, via *hyperthreading*), a eficiência cai para cerca de 44% nas cargas média e grande.

Sumário

1	Introdução	2
2	Elementos comuns às duas implementações	3
2.1	Constantes simbólicas e tipos enumerados	3
2.2	Disposição da matriz em memória	3
2.3	Funções de cálculo compartilhadas	4
2.4	Leitura e validação da entrada	4
2.5	Preparação da simulação	4
3	Solução sequencial	5
3.1	Núcleo da simulação	5
3.2	Cálculo do índice linear	6
3.3	Complexidade	6

4	Solução paralela	7
4.1	Onde está o paralelismo	7
4.2	Região paralela persistente	7
4.3	Reduções e determinismo	8
4.4	Otimizações aplicadas	8
4.5	SIMD	9
5	Validação	10
5.1	Metodologia	10
5.2	Cargas de teste	10
5.3	Resultados da validação	11
5.4	Independência do número de <i>threads</i>	11
6	Ambiente experimental	11
7	Resultados de desempenho	12
7.1	Tempos de execução	12
7.2	Speedup e eficiência	13
7.3	Vazão por atualização de célula	13
7.4	Isolamento do ganho monothread	14
7.5	Varredura do número de <i>threads</i>	15
7.6	Comparação de <i>schedules</i>	16
8	Análise dos resultados	17
9	Conclusão	18
A	Guia de compilação e execução	19

1 Introdução

Este trabalho consiste no desenvolvimento e na análise de desempenho de uma simulação baseada em autômatos celulares para a propagação direcional de incêndios florestais. O modelo representa a área de estudo como uma matriz bidimensional discreta, na qual o estado de cada célula no instante seguinte é determinado a partir de seu estado atual, do estado de sua vizinhança de Moore (oito vizinhos), da cobertura do solo, da umidade da vegetação, da direção e intensidade de um vento constante, e da ativação programada de zonas de contenção.

A complexidade computacional desse tipo de simulação escala linearmente com o produto entre o número de células da matriz e o número de passos temporais executados, tornando cenários de grande escala computacionalmente custosos em execução puramente sequencial. Por outro lado, fixado um passo de tempo, o cálculo do próximo estado de cada célula é independente das demais, o que caracteriza um problema que possibilita paralelismo de dados em memória compartilhada.

Para emular o cenário físico, a simulação implementa três características:

- (i) Influência direcional imposta pelo vento, o qual favorece ou desfavorece a propagação do fogo conforme o alinhamento do vento;
- (ii) Heterogeneidade do terreno, dividida em quatro classes de cobertura com distintos fatores de combustibilidade e níveis de umidade;

- (iii) Zonas de contenção que convertem células intactas em barreiras permanentes em passos pré-determinados.

Considerando que as abordagens são submetidas às mesmas condições, as implementações sequencial e paralela devem produzir exatamente os mesmos resultados numéricos e estatísticos, diferindo apenas no tempo de execução.

Frente ao exposto, segue a organização do relatório. A Seção 2 descreve os elementos comuns às duas implementações. A Seção 3 descreve a implementação sequencial em C, enquanto a Seção 4 detalha a estratégia de paralelização com OpenMP e suas otimizações. A Seção 5 valida a equivalência entre as duas versões. A Seção 6 caracteriza o ambiente experimental, a Seção 7 exibe os tempos de execução, *speedup* e eficiência, a Seção 8 discute os resultados obtidos e, por fim, a Seção 9 traz as considerações finais.

2 Elementos comuns às duas implementações

As duas versões compartilham o cabeçalho de definições, as estruturas de dados, as funções de cálculo e toda a etapa de leitura, validação e preparação. Essas partes são descritas aqui uma única vez, de modo que as Seções 3 e 4 tratem apenas daquilo que distingue cada versão. Ambos os arquivos seguem a mesma organização em três blocos: um cabeçalho de definições, um conjunto de funções de cálculo e de leitura, e a função `main`, que orquestra preparação, simulação e apresentação dos resultados.

2.1 Constantes simbólicas e tipos enumerados

A parte inicial dos programas compreende os valores imutáveis do enunciado, definidos a partir de constantes e de enumerações no código. Por exemplo, os oito vizinhos de Moore são declarados como um vetor constante de deslocamentos:

```
1 typedef struct { int vertical; int horizontal; } DIRECAO;
2
3 static const DIRECAO VIZINHOS[8] = {
4     DIRECAO_NORTE,     DIRECAO_NORDESTE,
5     DIRECAO_LESTE,     DIRECAO_SUDESTE,
6     DIRECAO_SUL,       DIRECAO_SUDOESTE,
7     DIRECAO_OESTE,     DIRECAO_NOROESTE
8 };
```

Listing 1: Vizinhança de Moore como vetor de deslocamentos.

2.2 Disposição da matriz em memória

A matriz é representada por sete vetores de valores inteiros armazenados de forma contígua, agrupados em uma única `struct CELULAS` (Listagem 2): `cobertura`, `umidade`, `estado_atual`, `tempo_atual`, `proximo_estado`, `proximo_tempo` e `ativacao`.

```
1 typedef struct {
2     int *cobertura;
3     int *umidade;
4     int *estado_atual;
5     int *tempo_atual;
6     int *proximo_estado;
7     int *proximo_tempo;
8     int *ativacao;
9 } CELULAS;
```

Listing 2: Estrutura de dados da grade (*Structure of Arrays*).

A organização em *Structure of Arrays* mantém cada atributo das células em um vetor separado e contíguo na memória. Dessa forma, quando um determinado atributo é percorrido, como `estado_atual`, os valores utilizados estão armazenados sequencialmente, favorecendo o uso da memória cache e evitando carregar, junto com eles, outros campos da célula que não são necessários naquele momento. Essa organização também simplifica a atualização entre passos da simulação. Como os campos são ponteiros para vetores distintos, ao final de cada passo não é necessário copiar os $L \cdot C$ elementos calculados: basta trocar os ponteiros dos vetores atual e próximo, fazendo com que essa operação tenha custo constante, $O(1)$.

Na versão paralela, os laços percorrem esses vetores em blocos de índices, o que também reduz a ocorrência de falso compartilhamento. Esse problema ocorre quando *threads* diferentes escrevem em posições distintas que pertencem à mesma linha de cache, causando invalidações desnecessárias entre os núcleos. Com `schedule(static)`, os índices são divididos em blocos contíguos entre as *threads*, fazendo com que esse efeito possa ocorrer principalmente nas fronteiras entre os blocos. Nos experimentos com `dynamic` e `guided`, foi utilizado um *chunk* de 1024 elementos, de forma que o trabalho também é distribuído em blocos contíguos relativamente grandes, reduzindo a frequência dessas fronteiras. Em contrapartida, vale destacar que uma política como `schedule(static, 1)` distribuiria células consecutivas entre *threads* diferentes, aumentando significativamente a possibilidade de falso compartilhamento e, por esse motivo, essa configuração não foi utilizada.

2.3 Funções de cálculo compartilhadas

Os cálculos básicos do modelo foram organizados em funções auxiliares mantidas de forma idêntica nas versões sequencial e paralela. A função `fator_cobertura` associa cada tipo de cobertura ao seu fator de combustível, enquanto `tempo_queima_inicial` determina por quantos passos uma célula permanece em chamas. A contribuição de cada vizinho para a propagação do incêndio é calculada por `peso_vizinho`, que considera sua posição relativa e a influência do vento. A partir da soma dessas contribuições, `potencial_ignicao` calcula o potencial de ignição da célula considerando também seu tipo de cobertura e sua umidade. Por fim, `percentual_queimado` e `percentual_protegido` calculam as métricas apresentadas ao término da simulação.

Essas funções recebem todos os valores necessários por parâmetro e não dependem de estado global, o que permite utilizar a mesma lógica de cálculo nas duas implementações. Dessa forma, as principais diferenças entre as versões sequencial e paralela ficam concentradas na forma como a grade é percorrida, na distribuição do trabalho entre as *threads*, na acumulação das estatísticas e na sincronização necessária entre as etapas de cada passo.

2.4 Leitura e validação da entrada

A leitura do arquivo de entrada foi dividida de acordo com os diferentes conjuntos de parâmetros da simulação. Cada etapa realiza também as validações correspondentes, permitindo interromper a execução assim que uma entrada inválida é identificada. São verificados, entre outros aspectos, as dimensões da matriz, os parâmetros do vento, a quantidade e a posição dos focos iniciais e os limites das zonas de contenção. Também são rejeitados focos repetidos ou posicionados sobre células não combustíveis, como água e solo exposto.

2.5 Preparação da simulação

Após a leitura das configurações gerais, a grade é inicializada pela função de geração do terreno. As células são percorridas uma única vez em ordem crescente de índice. Para cada uma, primeiro é determinada a cobertura e, em seguida, sua umidade, conforme apresentado na Listagem 3. Essa ordem é mantida porque ambos os valores são obtidos por chamadas sucessivas a `rand_r`; assim, uma mesma *seed* sempre produz a mesma configuração inicial da grade.

```

1  for (long long i = 0; i < total_celulas; i++) {
2
3      '''
4      int val = rand_r(&seed) % 100;
5
6      if      (val <= COBERTURA_MAX_AGUA)      { /* agua, nao combustivel */ }
7      else if (val <= COBERTURA_MAX_SOLO)      { /* solo, nao combustivel */ }
8      else if (val <= COBERTURA_MAX_RASTEIRA) { /* rasteira, intacta */ }
9      else                                     { /* floresta, intacta */ }
10
11      g->umidade[i]      = rand_r(&seed) % 101;
12      g->tempo_atual[i] = 0;
13      g->ativacao[i]     = CELULA_SEM_CONTENCAO;
14      '''
15
16  }

```

Listing 3: Geração do terreno e da umidade, idêntica nas duas versões.

Depois da geração do terreno, os focos iniciais são aplicados às células indicadas na entrada. Em seguida, é construído o mapa de contenção, no qual cada posição armazena o passo em que a contenção deve ser ativada. Quando uma célula pertence a mais de uma zona, é mantido o menor passo de ativação entre elas.

Toda essa preparação ocorre antes do início da medição de tempo. Nessa etapa também é realizada uma primeira varredura da grade para contabilizar a quantidade inicial de células combustíveis e a distribuição dos estados. Na implementação sequencial, essa contagem é feita por uma varredura simples da grade, enquanto a versão paralela utiliza um `omp parallel for` com reduções sobre os respectivos contadores. Como essa operação ocorre antes do trecho cronometrado em ambas as versões, ela não é considerada na comparação dos tempos de execução da simulação.

3 Solução sequencial

A versão sequencial utiliza as estruturas de dados, as funções auxiliares e as etapas de preparação apresentadas na Seção 2. Nesta seção são descritos apenas os aspectos específicos de `fire_seq.c`, com destaque para o funcionamento do laço principal, algumas decisões de implementação e sua complexidade.

3.1 Núcleo da simulação

A simulação é executada enquanto ainda houver células em chamas e o número máximo de passos não tiver sido atingido. Essas duas condições são verificadas diretamente no laço principal:

```

1  while (passo_atual < P && cnt.em_chamas > 0) {
2      ...
3  }

```

A variável `passo_atual` indica o passo corrente da simulação, enquanto `cnt.em_chamas` armazena a quantidade de células que permanecem em chamas. Antes da primeira iteração, esse valor é obtido na varredura inicial descrita na Seção 2.5. Dessa forma, caso não existam focos ativos após a inicialização, nenhum passo é executado.

Cada passo da simulação realiza duas varreduras principais sobre a grade. A primeira verifica quais zonas de contenção devem ser ativadas naquele momento e altera o estado das células intactas correspondentes. A segunda calcula o estado de todas as células para o passo seguinte e, simultaneamente, atualiza as estatísticas da simulação.

O tratamento de cada célula depende de seu estado atual, conforme resumido na Listagem 4. Células não combustíveis, queimadas ou pertencentes a zonas de contenção mantêm seu estado. Para células em chamas, o tempo restante de queima é decrementado e, quando chega a zero, a célula passa ao estado queimado. Já as células intactas exigem o maior volume de processamento, pois é necessário verificar seus vizinhos em chamas, calcular a soma de suas contribuições e determinar se o potencial de ignição é suficiente para iniciar a queima.

```

1 ESTADO_CODIGO estado_celula = (ESTADO_CODIGO)celulas.estado_atual[i];
2
3 if (estado_celula == ESTADO_INTACTA) {
4     /* calcula S e o potencial de ignicao,
5        decidindo entre intacta e em chamas */
6
7 } else if (estado_celula == ESTADO_EM_CHAMAS) {
8     /* decrementa o tempo restante de queima;
9        ao chegar a zero, passa para queimada */
10
11 } else {
12     /* NAO_COMBUSTIVEL, QUEIMADA ou CONTENCAO:
13        permanece no mesmo estado */
14 }

```

Listing 4: Estrutura da cadeia `if/else if` de atualização de estado em `fire_seq.c`.

Durante essa varredura, os resultados são escritos em `proximo_estado` e `proximo_tempo`, enquanto `estado_atual` e `tempo_atual` permanecem apenas como fonte dos dados do passo corrente. Essa separação garante que a atualização de uma célula não interfira no cálculo das demais durante o mesmo passo.

Ao final da iteração, os vetores atuais e futuros são trocados por meio da permutação de seus ponteiros. Assim, os resultados calculados passam a representar o estado corrente da simulação sem que seja necessário copiar os $L \cdot C$ elementos entre os vetores. A troca possui, portanto, custo constante, $O(1)$.

Após o término da simulação, o *checksum* e os percentuais de área queimada e protegida são calculados fora do trecho cronometrado, conforme definido para ambas as implementações.

3.2 Cálculo do índice linear

Embora a grade represente uma matriz de L linhas e C colunas, seus dados são armazenados em vetores lineares. Durante a varredura principal, os laços utilizam diretamente as coordenadas de linha e coluna, e o índice correspondente é obtido por

$$i = l \cdot C + c.$$

Essa abordagem evita converter repetidamente um índice linear em coordenadas bidimensionais por meio de operações de divisão e módulo. Uma versão anterior realizava essa conversão para cada célula processada, utilizando expressões equivalentes a i / C e $i \% C$. Como linha e coluna já estão disponíveis nos dois laços aninhados, o cálculo direto elimina essas operações do caminho principal da simulação.

Além de simplificar o acesso à grade, essa organização também mantém a estrutura dos laços compatível com a versão paralela, na qual as dimensões da matriz são utilizadas diretamente na distribuição das iterações entre as *threads*.

3.3 Complexidade

Em cada passo são realizadas duas varreduras sobre as $L \cdot C$ células da grade. A ativação das zonas possui custo $O(L \cdot C)$ e a atualização dos estados também percorre toda a matriz.

Para cada célula intacta, são examinados no máximo oito vizinhos, quantidade constante que não altera a ordem assintótica do algoritmo.

Consequentemente, o custo de um passo é

$$O(L \cdot C),$$

e, considerando no máximo P passos, a complexidade temporal da simulação é

$$O(P \cdot L \cdot C).$$

O consumo de memória também é $O(L \cdot C)$, uma vez que a representação da grade utiliza um número constante de vetores com um elemento para cada célula.

4 Solução paralela

4.1 Onde está o paralelismo

O ponto de partida da paralelização é reconhecer onde ela *não* é possível. O estado do passo $p+1$ depende de todo o estado do passo p , inclusive de células arbitrariamente distantes através da cadeia de propagação. Não existe, portanto, paralelismo entre passos: há uma barreira lógica por passo, e o número de passos é um limite duro de serialização.

Fixado um passo, por outro lado, o cálculo das $L \cdot C$ células do próximo estado é completamente independente. Todas as *threads* leem exclusivamente de `estado_atual`, `tempo_atual`, `cobertura` e `umidade`, e escrevem exclusivamente em `proximo_estado` e `proximo_tempo`. Não há escrita em posição lida por outra *thread* no mesmo passo, o que elimina por construção a condição de corrida sobre a matriz, sem qualquer sincronização explícita dentro da varredura.

4.2 Região paralela persistente

Uma implementação possível seria abrir uma região paralela a cada passo, pagando P vezes o custo de criação e destruição do time de *threads*. No entanto, a versão implementada abre a região paralela uma única vez, *antes* do laço de passos, e todas as *threads* executam o `while` redundantemente:

```

1  #pragma omp parallel num_threads(T) default(none) \
2      shared(L, C, P, total_celulas, celulas, LIMIAR, passo_atual, cnt, pico, \
3          deslocamento_offset, pesos_direcao, vizinhos, \
4          proximo_celulas_em_chamas, ignicoes_no_passo, \
5          proximo_nao_combustiveis, \
6          proximo_intactas, proximo_queimadas, proximo_contencao)
7  {
8      while (passo_atual < P && cnt.em_chamas > 0) {
9          #pragma omp for schedule(SCHED) /* 1. ativacao das zonas */
10         for (long long i = 0; i < total_celulas; i++) { /* ... */ }
11
12         #pragma omp for collapse(2) schedule(SCHED) /* 2. proximo estado */
13         reduction(+:proximo_celulas_em_chamas, ignicoes_no_passo, ...)
14         for (int l = 0; l < L; l++)
15             for (int c = 0; c < C; c++) { /* ... */ }
16
17         #pragma omp single /* 3-5. troca e controle */
18         { /* swap de ponteiros, pico, passo_atual++ */ }
19     }
20 }

```

Listing 5: Estrutura da região paralela persistente (`fire_omp.c`).

A decisão por essa construção repousa nas barreiras implícitas do OpenMP. A barreira ao final do primeiro `omp for` garante que toda a ativação das zonas do passo corrente esteja concluída antes de qualquer *thread* começar a ler `estado_atual` no cálculo do próximo estado. A barreira ao final do segundo `omp for` garante que todas as escritas em `proximo_estado` tenham ocorrido antes da troca de ponteiros, e a barreira ao final do `omp single` garante que os novos valores de `passo_atual` e `cnt.em_chamas` estejam visíveis a todas as *threads* quando a condição do `while` for reavaliada.

4.3 Reduções e determinismo

Os contadores que variam a cada passo são agregados por `reduction(+:...)` no próprio laço de atualização, o que dispensa `critical/atomic` no laço principal. Como os contadores são inteiros, a soma é associativa e exata, e o resultado da redução é rigorosamente independente da ordem de combinação das somas parciais das *threads*.

4.4 Otimizações aplicadas

Quatro otimizações foram aplicadas ao laço de atualização de estado por célula que, a cada passo, percorre as $L \times C$ células calculando o potencial de ignição e o próximo estado (Listagem 5 na versão paralela; Listagem 4 na sequencial). As quatro são **independentes da paralelização em si** e estão presentes, de forma idêntica, em `fire_seq.c` e em `fire_omp.c`. A única diferença entre os dois laços é a presença das diretivas `#pragma omp` em torno do mesmo código. Isso é relevante para a Seção 8: qualquer diferença de tempo entre as duas versões executando com uma única *thread* não pode vir dessas otimizações, já que ambas as compartilham.

Pré-cálculo dos pesos direcionais: O vento é constante durante toda a simulação, de modo que os oito pesos P_v e os oito deslocamentos lineares correspondentes são calculados uma única vez, antes do laço de passos.

Caminho rápido para o interior da matriz: Para células com $0 < \ell < L-1$ e $0 < c < C-1$, todos os oito vizinhos existem e os testes de limite são desnecessários.

Curto-circuito na avaliação do potencial: Ambas as versões evitam o cálculo de potencial para células intactas sem vizinho em *chamas*.

Colapso dos laços e índice linear direto: O laço de atualização é escrito como dois `for` aninhados sobre linha e coluna (com `collapse(2)` na versão paralela), e o índice linear de cada célula é calculado diretamente como `i = (long long)l * C + c`, sem divisão/módulo (Seção 3.2).

Reescrita *branchless* da ativação de zonas e da soma dos vizinhos: Diferentemente das outras três, esta otimização foi aplicada às duas versões por um motivo metodológico, não só de desempenho: as duas reescritas (Seção 4.5) inicialmente existiam inicialmente só em `fire_omp.c`, para viabilizar `#pragma omp simd`. Foi decidido migrá-las também para `fire_seq.c`, sem nenhuma diretiva `omp`, para eliminar essa diferença de forma do código-fonte entre as duas versões, de modo que o speedup medido na Seção 7 reflita estritamente o ganho da paralelização em si, e não uma mistura entre paralelização e otimização serial presente em só uma das implementações.

4.5 SIMD

A versão paralela usa duas diretivas `simd` explícitas, em pontos escolhidos justamente por serem varreduras curtas e sem controle de fluxo dependente de dado por iteração; o restante do laço principal foi deixado fora do escopo do `simd` pelos motivos discutidos a seguir.

Onde foi aplicado. *Ativação das zonas de contenção* (`#pragma omp for simd schedule(SCHED)`, dentro da região persistente da Listagem 5): o corpo do laço é reescrito em forma *branchless*. Em vez de pular a escrita quando a célula não deve ser ativada, o laço sempre escreve, selecionando entre o novo estado e o estado atual com um operador ternário:

```
1 int deve_ativar = (celulas.ativacao[i] == passo_atual) & (celulas.estado_atual[i]
  ] == ESTADO_INTACTA);
2 celulas.estado_atual[i] = deve_ativar ? ESTADO_CONTENCAO : celulas.estado_atual[
  i];
```

Listing 6: Ativação de zonas em forma *branchless*, vetorizável sem masked-store.

Reescrever o mesmo valor quando a condição é falsa não tem efeito algum sobre o resultado, e evita a necessidade de um *masked store* de hardware (disponível só a partir de AVX2): o compilador vetoriza a seleção com uma instrução de *blend*, que já existe no baseline x86-64 usado pelas flags padrão do Makefile (`-O2`, sem `-march=native`). O uso de `&` em vez de `&&` também é proposital: o `&&` de C introduz avaliação de curto-circuito, ou seja, controle de fluxo, o que bastaria para o vetorizador desistir do laço.

Soma dos vizinhos em chammas, caminho interno (`#pragma omp simd reduction(+:S)`, dentro do caminho rápido descrito na Seção 4.4): as oito iterações sobre os vizinhos de Moore são reescritas da forma condicional usual,

```
1 if (celulas.estado_atual[i + deslocamento_offset[k]] == ESTADO_EM_CHAMAS)
2   S += pesos_direcao[k];
```

Listing 7: Forma com desvio, não vetorizável.

para uma forma aritmética equivalente e sem desvio,

```
1 int atual_em_chamas = (celulas.estado_atual[i + deslocamento_offset[k]] ==
  ESTADO_EM_CHAMAS);
2 S += pesos_direcao[k] * atual_em_chamas;
```

Listing 8: Forma sem desvio usada em `fire_omp.c`, vetorizável.

que multiplica cada peso por 0 ou 1 em vez de somá-lo condicionalmente. Como o laço tem exatamente oito iterações fixas, sem dependência entre elas além da própria redução em `S` (já expressa pela cláusula `reduction(+:S)`), o compilador pode somar os oito produtos em paralelo dentro de um único registrador vetorial.

Onde não foi aplicado. *O laço externo de atualização* (`#pragma omp for collapse(2)`, Listagem 5) não recebeu `simd`. Ele contém um desvio por estado da célula (não combustível, intacta, em chammas, queimada, contenção) e, dentro do caso intacta, um segundo desvio que escolhe entre o caminho rápido do interior e o caminho genérico de borda (Seção 4.4). Diferente da ativação de zonas, aqui os ramos não escrevem o mesmo tipo de valor por trás de uma seleção barata: cada estado grava um `proximo_tempo` diferente, e o próprio curto-circuito quando `S = 0` (Seção 4.4) existe exatamente para *pular* trabalho. Forçar esse laço inteiro para uma forma *branchless*, ao estilo da Listagem 6, eliminaria esse curto-circuito e obrigaria a calcular o potencial de ignição para toda célula intacta em toda iteração, trocando uma otimização já comprovada por uma vetorização de retorno incerto. Testado com `-fopt-info-vec-optimized`, o gcc de fato recusa esse laço com `missed: not vectorized: control flow in loop` nas flags do Makefile.

O *caminho de borda* (o `else` da soma de vizinhos, usado só pelas células do perímetro da matriz) também ficou de fora. Além de já ser o ramo alternativo de um desvio (o que por si só desqualifica o laço externo para `simd`), ele testa os limites da matriz individualmente para cada um dos oito vizinhos, com um número de vizinhos válidos que varia por célula, o oposto do caso interno, em que os oito deslocamentos são sempre válidos e a soma é uma redução de tamanho fixo. Como o perímetro é $O(L + C)$ células contra $O(L \cdot C)$ no total, o ganho possível de vetorizar esse caminho seria marginal frente à complexidade de reescrevê-lo sem desvios.

A *varredura inicial* de contagem de estados e coberturas (Seção 2), embora paralelizada com `reduction`, também não recebeu `simd`: além de envolver duas cadeias de `if/else` por célula (uma para cobertura, outra para estado), ela fica inteiramente fora do trecho cronometrado (Seção 2), de modo que vetorizá-la não teria efeito algum sobre os tempos reportados na Seção 7.

5 Validação

5.1 Metodologia

A validação compara, para cada entrada, as onze primeiras linhas da saída de `fire_seq` e `fire_omp`, excluindo apenas o campo `tempo`.

```

1 for f in entradas/*.txt; do
2   ./fire_seq "$f" | grep -v '^tempo' > /tmp/s.out
3   ./fire_omp "$f" | grep -v '^tempo' > /tmp/o.out
4   diff -q /tmp/s.out /tmp/o.out > /dev/null \
5     && echo "[OK]    $f" || echo "[FAIL] $f"
6 done

```

Listing 9: Script de validação.

5.2 Cargas de teste

Carga	$L \times C$	Células	P	LIMIAR	Vento (V)	F	Z	Atualizações
Pequena	400×500	200 000	80	35	leste (3)	3	2	16 M
Média	1200×1500	1 800 000	100	35	sudeste (2)	6	4	180 M
Grande	2500×2500	6 250 000	100	35	nordeste (4)	10	6	625 M

Tabela 1: Cargas utilizadas na validação e nos experimentos.

5.3 Resultados da validação

Campo	Pequena	Média	Grande
passos	80	100	100
nao_combustiveis	39 704	360 809	1 251 181
intactas	127 900	1 324 373	4 804 870
em_chamas	2 069	5 734	7 434
queimadas	28 105	93 801	121 067
contencao	2 222	15 283	65 448
total_ignicoes	30 171	99 529	128 491
pico_ignicoes	79 682	98 1 863	99 2 381
percentual_queimado	18,82	6,92	2,57
percentual_protegido	1,39	1,06	1,31
checksum	17073017104646724864	17187382406529706254	14825089465781038537
Resultado	[OK]	[OK]	[OK]

Tabela 2: Saída das versões sequencial e paralela.

5.4 Independência do número de *threads*

A Tabela 3 verifica diretamente o requisito de independência do resultado em relação ao número de *threads* (Seção 4): mesma entrada de carga média, variando $T \in \{1, 2, 4, 8, 16\}$, com *total_ignicoes*, *pico_ignicoes* e *checksum* idênticos em todos os casos.

T	Total ignições	Pico	Checksum
1	99 529	98 1 863	17187382406529706254
2	99 529	98 1 863	17187382406529706254
4	99 529	98 1 863	17187382406529706254
8	99 529	98 1 863	17187382406529706254
16	99 529	98 1 863	17187382406529706254

Tabela 3: Saída da versão paralela em função do número de *threads* (carga média).

6 Ambiente experimental

Os experimentos de desempenho foram executados no cluster fornecido pelo professor.

Item	Valor
Modelo do processador	Intel® Core™ i7-4790 @ 3,60 GHz
Núcleos físicos / lógicos	4 físicos / 8 lógicos (<i>Hyperthreading</i> , 2 <i>threads</i> /núcleo)
Cache L1 (d/i)	32 KiB + 32 KiB por núcleo (128 KiB + 128 KiB no total)
Cache L2	256 KiB por núcleo (1 MiB no total)
Cache L3	8 MiB (compartilhado)
Topologia NUMA	1 nó, 8 CPUs lógicas (sem penalidade de acesso remoto)
Memória RAM	31 GiB
Sistema operacional	Ubuntu 24.04.4 LTS
Compilador	gcc 5.3.1 (Ubuntu 5.3.1-14ubuntu2)
Especificação OpenMP suportada	4.0 (_OPENMP 201307)
Flags de compilação	-Wall -Wextra -O2 -fopenmp -std=c99

Tabela 4: Configuração do ambiente de execução.

As informações da Tabela 4 foram coletadas diretamente no ambiente de execução, com os comandos `lscpu` (modelo do processador, núcleos físicos e lógicos, cache), `cat /etc/os-release` (sistema operacional), `gcc -version` (versão do compilador), `gcc -fopenmp -dM -E - < /dev/null | grep -i openmp` (versão da especificação OpenMP suportada pelo compilador), `free -h` (memória disponível), `nproc -all` (contagem de unidades lógicas) e `numactl -hardware` (topologia NUMA, quando disponível).

7 Resultados de desempenho

7.1 Tempos de execução

A Tabela 5 reporta o tempo do núcleo cronometrado (Seção 2) para as três cargas de teste, cada uma executada com o número de *threads* T especificado no seu próprio arquivo de entrada (Tabela 1), e não escolhido livremente para esta comparação; por isso a carga pequena usa $T = 4$ enquanto as cargas média e grande usam $T = 8$. A comparação controlada do efeito de T , com a carga fixa, é tratada separadamente na Seção 7.5. O núcleo paralelo é de 3,29 a $3,54\times$ mais rápido que o sequencial nas três cargas (Tabela 6), um valor praticamente constante entre cargas que diferem em quase $40\times$ em número de atualizações, consistente com um ganho de paralelização proporcional ao volume de trabalho por célula, e não com algum custo fixo por passo (como overhead de sincronização) que pesasse proporcionalmente mais nas cargas menores. Esse ganho bruto já não inclui nenhuma inflação serial relevante: a Seção 7.4 mostra que o fator monothread entre as duas implementações está hoje próximo de 1, de modo que o ganho bruto medido aqui é, ele mesmo, quase todo atribuível à paralelização.

Carga	Atualizações	T	Sequencial (s)	Paralela (s)
Pequena	16 M	4	0,153715	0,043488
Média	180 M	8	1,780002	0,541490
Grande	625 M	8	6,182823	1,878302

Tabela 5: Tempo de execução do núcleo da simulação, em segundos.

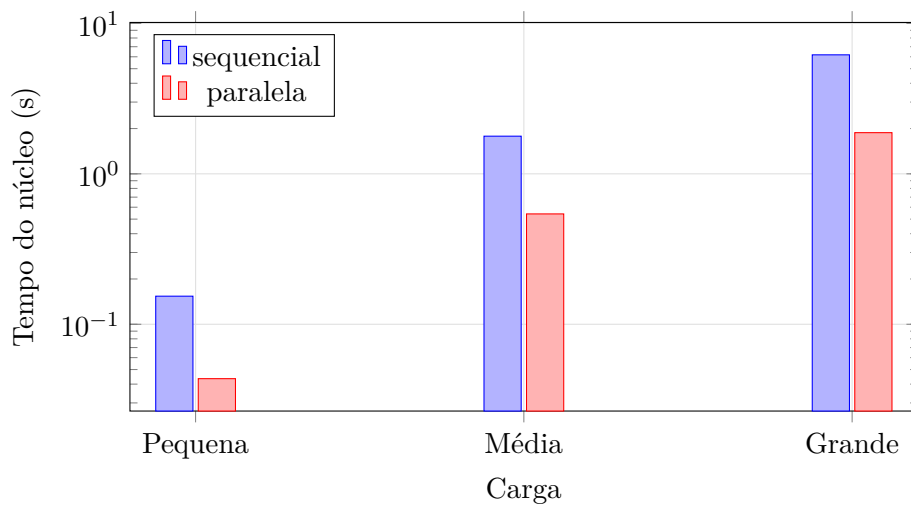


Figura 1: Tempo do núcleo da simulação nas duas versões, em escala logarítmica.

7.2 Speedup e eficiência

O speedup bruto $S(T) = t_{\text{seq}}/t_{\text{par}}$ varia de 3,29 a 3,54 $\times$ (Tabela 6), abaixo do número nominal de *threads* usado em cada caso ($T = 4$ ou $T = 8$). A eficiência bruta correspondente, $E(T) = S(T)/T$ usando o T nominal como denominador, já é sublinear: 0,88 na carga pequena e 0,41 nas cargas média e grande. Essa eficiência bruta não precisa necessariamente ser corrigida por um fator monothread expressivo: a Seção 7.4 mostra que esse fator está hoje próximo de 1, de modo que a eficiência puramente paralela discutida ali muda pouco em relação à eficiência bruta aqui reportada.

Carga	T	$S(T)$	$E(T)$
Pequena	4	3,535	0,884
Média	8	3,287	0,411
Grande	8	3,292	0,411

Tabela 6: Speedup e eficiência medidos.

7.3 Vazão por atualização de célula

A vazão sequencial é praticamente constante entre as três cargas (104,1, 101,1 e 101,1 milhões de atualizações por segundo, Tabela 7), confirmando empiricamente o comportamento $O(L \cdot C \cdot P)$ discutido na Seção 3. Já a vazão paralela por *thread* (92,0 na carga pequena; 41,6 nas outras duas) fica *abaixo* da vazão sequencial por célula, não acima dela. Isso é o esperado pois as bases do código são praticamente idênticas (Seção 7.4): a vazão por *thread* reflete só o paralelismo real obtido, sem nenhuma vantagem adicional herdada de otimizações presentes exclusivamente na versão paralela ou na versão sequencial; a razão entre vazão paralela por *thread* e vazão sequencial coincide, célula a célula, com a eficiência bruta $E(T)$ já reportada na Seção 7.1.

Carga	T	Sequencial	Paralela	Paralela por <i>thread</i>
Pequena	4	104,1	367,9	92,0
Média	8	101,1	332,4	41,6
Grande	8	101,1	332,7	41,6

Tabela 7: Vazão em milhões de atualizações de célula por segundo.

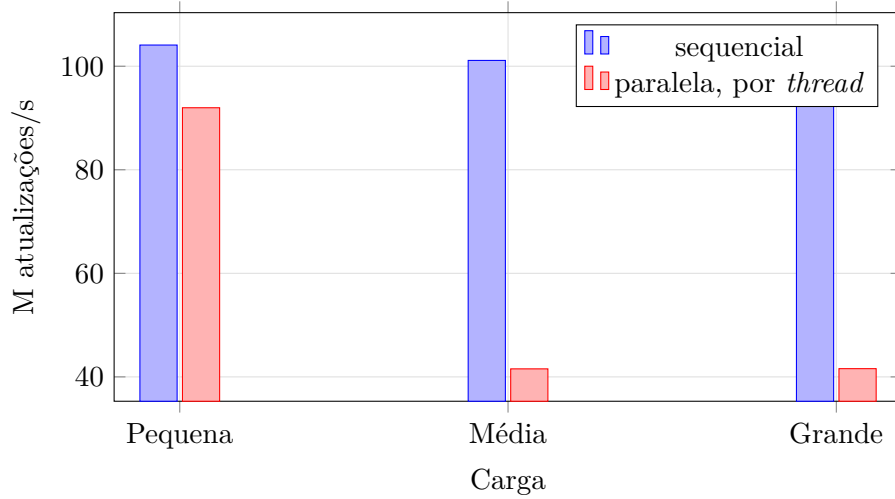


Figura 2: Vazão sequencial e vazão paralela por *thread*, em milhões de atualizações de célula por segundo. A vazão paralela por *thread* maior que a sequencial, e não apenas igual a ela, é o mesmo sinal do fator monothread isolado na Seção 7.4.

7.4 Isolamento do ganho monothread

Para isolar qualquer efeito puramente estrutural entre as duas implementações (por exemplo, o próprio custo de entrar em uma região `omp parallel`, ainda que com uma única *thread*) do efeito da paralelização em si, a versão paralela foi executada com $T = 1$ e comparada contra a versão sequencial, ambas fixadas em um único núcleo lógico com `taskset -c 0`. Com uma única *thread* nos dois casos, nenhuma das duas versões executa qualquer trabalho em paralelo, de modo que toda diferença de tempo entre elas é necessariamente de origem serial. As cinco otimizações da Seção 4.4 (pré-cálculo dos pesos, caminho rápido do interior, curto-circuito, índice linear direto e a reescrita *branchless* da ativação de zonas e da soma dos vizinhos) são hoje idênticas nas duas versões; a única diferença de código-fonte entre `fire_seq.c` e `fire_omp.c` nesses trechos são as diretivas `#pragma omp` em torno deles, ausentes por definição no sequencial. Em princípio, as duas diretivas `simd` explícitas da Seção 4.5 poderiam valer mesmo com $T = 1$ (`#pragma omp simd` é uma instrução de vetorização, não de paralelismo entre *threads*), mas, como discutido a seguir, os resultados mostram que elas não se traduzem em nenhuma vantagem de tempo mensurável neste hardware e flags de compilação. Essa comparação foi feita apenas para as cargas pequena e média, por conveniência experimental; a carga grande é tratada de forma equivalente, sem `taskset`, na varredura de T com $T = 1$ (Seção 7.5).

Carga	Sequencial (s)	Paralela, $T = 1$ (s)	Razão
Pequena	0,157827	0,162695	0,970
Média	1,781567	1,900929	0,937

Tabela 8: Ganho monothread: versão paralela com $T = 1$ contra a versão sequencial.

A razão obtida, $0,970\times$ na carga pequena e $0,937\times$ na carga média, é chamada aqui de *fator monothread*: o ganho serial da versão paralela sobre a sequencial, medido sem nenhum paralelismo real envolvido. A versão paralela com $T = 1$ é, na prática, tão rápida quanto a sequencial na carga pequena, e cerca de 6% mais lenta na carga média, não mais rápida. Como os laços com as diretivas *branchless* são curtos, e de tamanho fixo pequeno (oito iterações), é possível que a diretiva explícita da versão paralela simplesmente não traga vantagem adicional notável. A pequena diferença residual observada (a versão paralela com $T = 1$ nunca é mais

rápida, e é até um pouco mais lenta na carga média) é mais bem explicada pelo custo fixo, ainda que pequeno, de entrar em uma região `omp parallel` e sincronizar via `omp single` a cada passo (Seção 4) mesmo com uma única *thread* ativa, do que por qualquer otimização serial ausente. Essa leitura é corroborada de forma independente pela varredura de T na carga grande (Seção 7.5): com $T = 1$, $S(T) = 0,94$, no mesmo sentido e na mesma ordem de grandeza, sem depender do isolamento por `taskset` usado nesta seção.

Carga	T	S medido	Fator monothread	S paralelo	E paralela ($/T$ nominal)	E paralela ($/4$ núcleos físicos)
Pequena	4	3,53	0,970	3,64	0,91	0,91
Média	8	3,29	0,937	3,51	0,44	0,88
Grande	8	3,29	$\approx 0,94$	$\approx 3,51$	$\approx 0,44$	$\approx 0,88$

Tabela 9: Decomposição do speedup medido em ganho monothread e ganho paralelo, com eficiência calculada contra o número nominal de *threads* e contra os núcleos físicos reais.

A Tabela 9 decompõe o speedup medido dividindo-o pelo fator monothread, $S_{\text{par}} = S_{\text{medido}}/\text{fator monothread}$, isolando o ganho atribuível apenas aos múltiplos núcleos. Como o fator está próximo de 1 (Seção 7.4), essa correção é pequena: S_{par} fica apenas 3 a 7% acima de S_{medido} . Como a carga grande não passou pelo isolamento monothread da Tabela 8 (nota na Seção 7.4), seu fator é extrapolado a partir da carga média e marcado com $\approx$, extrapolação essa corroborada pela medição direta e independente de $S(T = 1) = 0,94$ na varredura de T para essa mesma carga (Seção 7.5). A tabela reporta duas eficiências para S_{par} : dividida pelo número nominal de *threads* T do arquivo de entrada, e dividida pelos 4 núcleos físicos reais do processador (Tabela 4). A primeira cai de 91% na carga pequena ($T = 4$, igual ao número de núcleos físicos) para 44% nas cargas média e grande ($T = 8$, o dobro dos núcleos físicos); a segunda, normalizada contra o paralelismo físico real disponível, fica em 88–91% nas três cargas, próxima do linear. As duas leituras não são contraditórias: a primeira mostra que a segunda metade das 8 *threads* nominais (as lógicas, via *hyperthreading*) contribui pouco além do que os 4 núcleos físicos já entregam sozinhos; a segunda mostra que a estratégia de paralelização em si (região persistente, `omp for collapse(2)`, reduções) pode estar próxima do ótimo frente ao paralelismo físico disponível. A leitura relevante para avaliar a qualidade da paralelização implementada é a segunda, e é essa a retomada na Seção 8.

7.5 Varredura do número de *threads*

T	Tempo (s)	$S(T)$	$E(T)$	$e(T)$
1	6,602237	0,94	0,94	—
2	3,495388	1,77	0,88	0,13
4	2,004134	3,09	0,77	0,10
8	1,883311	3,28	0,41	0,21
16	1,892489	3,27	0,20	0,26

Tabela 10: Tempo, speedup, eficiência e métrica de Karp–Flatt em função de T (carga grande).

O tempo cai de forma monotônica até $T = 8$ (1,88s) e praticamente estagna entre $T = 8$ e $T = 16$ (1,89s). O speedup acompanha esse padrão: parte de $S = 0,94$ em $T = 1$ (levemente abaixo de 1, o mesmo sinal do fator monothread próximo de 1 discutido na Seção 7.4), cresce até um pico de $S = 3,28$ em $T = 8$ e cai ligeiramente para $S = 3,27$ em $T = 16$. A eficiência bruta $E(T) = S(T)/T$ cai continuamente com T (de 0,94 em $T = 1$ para 0,88, 0,77, 0,41 e 0,20 em $T = 2, 4, 8$ e 16, respectivamente), com a queda mais acentuada exatamente na transição

de $T = 4$ para $T = 8$. Esse padrão é o esperado ao ultrapassar o número de núcleos físicos do processador (4, Tabela 4): *threads* adicionais além desse ponto disputam os mesmos núcleos via *hyperthreading*, entregando ganho marginal decrescente ($T = 8$) e depois praticamente nulo ($T = 16$) em vez de escalabilidade linear.

A Tabela 10 inclui também a métrica de Karp–Flatt $e(T) = (T/S_T - 1)/(T - 1)$, definida para $T \geq 2$, que reúne em um único número a fração serial efetiva — trechos sequenciais, custos de sincronização e desequilíbrio de carga — necessária para explicar o $S(T)$ observado. Dois padrões se destacam: $e(T)$ *decrece* de $T = 2$ (0,13) para $T = 4$ (0,10), indicando que mais *threads* amortizam os custos fixos de sincronização por passo e a fração serial efetiva cai; e *crece* de $T = 4$ para $T = 8$ (0,21) e $T = 16$ (0,26), sinal de que, ao ultrapassar os 4 núcleos físicos, o *hyperthreading* introduz contenção por recursos de execução dentro de cada núcleo, aumentando os *overheads* com p . Segundo a interpretação da disciplina, $e(p)$ crescente indica que “*overheads* ou desequilíbrios aumentam com p ”: é exatamente o que ocorre a partir de $T = 8$ neste processador.

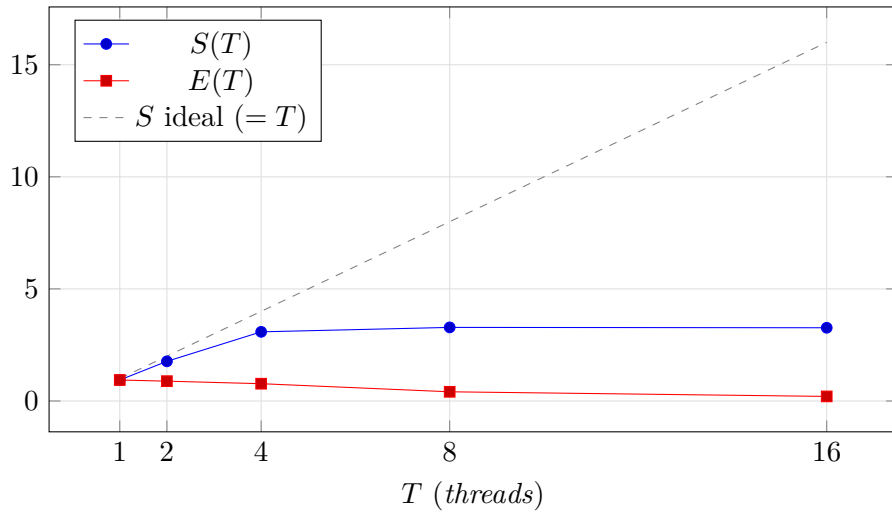


Figura 3: Speedup bruto $S(T)$ e eficiência bruta $E(T)$ em função do número de *threads*, carga grande, contra a reta de speedup ideal $S = T$. A curva de $S(T)$ acompanha $S = T$ até $T = 4$ (o número de núcleos físicos, Tabela 4) e se distancia dela a partir daí, sinal direto do limite do *hyperthreading* para esta carga de trabalho.

7.6 Comparação de *schedules*

<i>Schedule</i>	$T = 2$	$T = 4$	$T = 8$	$T = 16$
static	3,494086	2,002130	1,878103	1,895466
static, 1024	3,426964	1,956606	1,849952	1,883000
dynamic, 1024	3,673296	2,076709	1,930189	1,937384
guided	3,627628	2,065807	1,928899	2,041872

Tabela 11: Tempo de execução (s) para diferentes *schedules*, carga grande.

As quatro políticas de escalonamento produzem tempos relativamente próximos entre si em todo o intervalo de T testado, com uma diferença máxima de cerca de 8% entre a mais rápida e a mais lenta em qualquer T fixo (Tabela 11), o que indica baixo desbalanceamento de carga entre células: o corpo de trabalho por célula é majoritariamente uniforme (Seção 4), sem concentrações de trabalho que justifiquem escalonamento dinâmico. Interessantes notar que quem “vence” nos

quatro valores de T testados não é o `static` padrão, e sim `static,1024`, o mesmo `schedule` estático, mas com `chunk` fixo de 1024 elementos em vez do `chunk` contíguo padrão ($\approx L \cdot C/T$ elementos), por margem de 2 a 4% sobre o `static` padrão, que fica em segundo lugar; um `chunk` menor e fixo parece favorecer um pouco o aproveitamento de cache por `thread` nesta carga, sem pagar o custo de sincronização por `chunk` que `dynamic` paga a cada despacho. `dynamic,1024` e `guided` disputam as duas últimas posições sem um padrão único: `dynamic,1024` é a mais lenta em $T = 2$ e $T = 4$, `guided` em $T = 16$, e as duas ficam muito próximas em $T = 8$, o que, somado à margem estreita de `static,1024` sobre `static`, é consistente com uma diferença de desempenho pequena entre as quatro políticas, e não com uma política sistematicamente pior que as outras.

8 Análise dos resultados

O speedup bruto de 3,29 a 3,54× (Tabela 6) já é, ele mesmo, uma medida quase direta do paralelismo obtido. A Seção 7.4 isolou, comparando as duas versões com $T = 1$ em ambas (portanto sem nenhum trabalho paralelo real), um fator monothread de apenas 0,94 a 0,97×, ou seja, próximo de 1, e nunca a favor da versão paralela: com uma única `thread`, `fire_omp` é tão rápido quanto `fire_seq` na carga pequena e cerca de 6% mais lento na carga média, resultado corroborado de forma independente por $S(T=1) = 0,94$ na varredura de T da carga grande (Seção 7.5). Isso é consistente com o estado atual do código, onde todas as otimizações sequenciais implementadas têm forma textualmente idêntica em `fire_seq.c` e em `fire_omp.c`; só a presença das diretivas `#pragma omp` em torno delas muda entre os dois arquivos. Como os dois laços `branchless` são curtos (oito iterações fixas), o resultado sugere que a diretiva `simd` explícita não é uma fonte de ganho adicional. A pequena desvantagem residual da versão paralela com $T = 1$ é mais bem explicada pelo custo fixo de entrar em uma região `omp parallel` e sincronizar via `omp single` a cada passo (Seção 4), mesmo sem paralelismo real, do que por qualquer otimização serial que ainda falte na versão sequencial.

Descontado esse fator quase neutro, o ganho atribuível apenas à paralelização é $S_{\text{par}} \approx 3,51$ a 3,64× (Tabela 9), muito próximo do speedup bruto (a correção nunca ultrapassa 7%). A eficiência correspondente depende de qual contagem de `threads` se usa como referência: dividida pelos $T = 4$ ou $T = 8$ `threads` nominais de cada entrada, a eficiência cai de 91% (carga pequena, $T = 4$) para 44% (cargas média e grande, $T = 8$), uma queda expressiva. Dividida pelos 4 núcleos físicos reais do processador (Tabela 4), no entanto, a eficiência fica em 88 a 91% nas três cargas, próxima do linear. As duas leituras não são contraditórias, elas medem coisas diferentes. A primeira mostra que, ao usar 8 `threads` nominais em um processador com apenas 4 núcleos físicos, a segunda metade das `threads` (as lógicas, via `hyperthreading`) contribui muito pouco além do que os 4 núcleos físicos já entregam sozinhos, o comportamento esperado de `hyperthreading` em um laço com uso intenso de unidades aritméticas inteiras e pouca espera por memória. A segunda mostra que, contra o paralelismo físico real disponível, a estratégia de paralelização em si (região persistente, `omp for collapse(2)`, reduções) está próxima do ótimo. A leitura relevante para avaliar a qualidade da paralelização implementada é a segunda; a primeira é relevante para decidir quantas `threads` de fato vale a pena usar neste processador, e a resposta, à luz da varredura de T (Seção 7.5), é 4: o tempo praticamente estagna de $T = 4$ para $T = 8$ e $T = 16$, e a eficiência bruta despenca no mesmo ponto.

Lei de Amdahl e limite teórico do speedup. Isolando a fração serial f da equação de Amdahl $S(p) = 1/[f + (1 - f)/p]$ com a medição mais limpa disponível — $S(4) = 3,09$ na varredura da carga grande com $T = 4$, igual ao número de núcleos físicos e portanto sem interferência de `hyperthreading`:

$$f = \frac{p/S(p) - 1}{p - 1} = \frac{4/3,09 - 1}{3} \approx 9,8\%$$

O speedup máximo teórico com processadores ideais em número infinito é $S_{\max} = 1/f \approx 10,2\times$. Com $T = 8$ núcleos *físicos* hipotéticos (sem o custo do *hyperthreading*), Amdahl prevê $S(8) = 1/(0,098 + 0,902/8) \approx 4,3\times$; o valor medido de $3,28\times$ fica abaixo desse teto, diferença (4,3 vs 3,3) inteiramente explicada pelo *overhead* do *hyperthreading* quantificado pela métrica de Karp–Flatt (Seção 7.5). Nota-se também que o valor de f obtido via Amdahl coincide com o valor de $e(4)$ calculado pela Karp–Flatt (0,10): as duas fórmulas são algebricamente equivalentes quando a medição é livre de *overheads* externos ao código — confirmando a consistência entre os dois modelos.

Lei de Gustafson e capacidade de escalar o problema. Enquanto Amdahl fixa o tamanho do problema e pergunta “quanto mais rápido?”, Gustafson fixa o tempo e pergunta “quanto mais trabalho no mesmo tempo?”. O *speedup* escalado é $S^G(T) = \alpha + T(1 - \alpha)$, em que α é a fração serial de t_{par} . Usando a relação $\alpha = f \cdot S(T)$ com $f \approx 9,8\%$: para $T = 4$, $\alpha \approx 0,098 \times 3,09 \approx 0,30$ e $S^G(4) \approx 0,30 + 4 \times 0,70 = 3,1\times$ (idêntico ao Amdahl, como esperado quando f é constante); para $T = 8$, $\alpha \approx 0,098 \times 3,28 \approx 0,32$ e $S^G(8) \approx 0,32 + 8 \times 0,68 = 5,8\times$. Esse valor indica que, mantendo o tempo de execução paralela de 1,88s fixo, poderíamos processar um problema $5,8\times$ maior do que o que a versão sequencial resolve no mesmo tempo — perspectiva mais otimista do que os $3,28\times$ da leitura de Amdahl, pois Gustafson assume que a fração serial α não cresce com o tamanho do problema. Esse pressuposto é corroborado empiricamente: o *throughput* sequencial mantém-se praticamente constante nas três cargas testadas (101–104 Matualiz./s, Tabela 7), mostrando que a barreira de sincronização por passo (`omp single`) não escala com $L \times C$.

9 Conclusão

As implementações sequencial e paralela produziram resultados numericamente idênticos em todos os cenários testados, três cargas de trabalho e cinco valores de T , confirmando que a paralelização por dados dentro de cada passo, sem quebra de dependência temporal entre passos, preserva exatamente a semântica do modelo sequencial (Seção 5). A estratégia de região paralela persistente, `omp for collapse(2)` e reduções inteiras eliminou a necessidade de sincronização explícita (`critical/atomic`) no laço principal, apoiando-se apenas nas barreiras implícitas do OpenMP para a correção da troca de *buffers* e da condição de parada (Seção 4).

A decomposição do speedup bruto (Seção 8) mostrou que, com o código em seu estado atual, o fator monothread entre as duas versões está próximo de 1 (0,94 a 0,97 $\times$). Como resultado, o speedup bruto medido já é, essencialmente, o próprio ganho atribuível aos múltiplos núcleos, sem precisar de uma correção expressiva. Esse ganho é consistente com a arquitetura de 4 núcleos físicos do ambiente de testes (Tabela 4): a eficiência paralela normalizada por esses 4 núcleos fica em 88–91%, ao passo que a mesma eficiência normalizada pelas 8 *threads* nominais do processador cai para 44% nas cargas média e grande, evidenciando o limite prático do *hyperthreading* para esta carga de trabalho. A comparação entre políticas de escalonamento (`static, static, 1024, dynamic, 1024, guided`) mostrou diferenças moderadas entre si (no máximo cerca de 8%) em todo o intervalo de T testado, com `static, 1024` sendo a opção mais rápida em todos os casos testados, por margem estreita sobre o `static` padrão; ainda assim, a diferença pequena entre as quatro políticas indica que a carga de trabalho por célula é suficientemente uniforme para que o escalonamento estático seja uma escolha adequada, sem necessidade de balanceamento dinâmico. A análise complementar pela Lei de Amdahl (fração serial $f \approx 9,8\%$, $S_{\max} \approx 10,2\times$), pela Lei de Gustafson ($S^G(8) \approx 5,8\times$ para problema escalado) e pela métrica de Karp–Flatt ($e(T)$ decrescente de $T = 2$ a $T = 4$ e crescente a partir de $T = 8$) converge para o mesmo diagnóstico: a implementação está próxima do ótimo para o paralelismo físico disponível, e o gargalo dominante é o limite de 4 núcleos físicos do processador, não a qualidade da paralelização em si.

A Guia de compilação e execução

Esta seção resume o necessário para compilar e executar as duas versões a partir dos três arquivos entregues (`fire_seq.c`, `fire_omp.c` e `Makefile`), seguindo a forma de invocação exigida no enunciado (Seção 3).

```
1 make all      # compila as duas versoes: gera os binarios fire_seq e fire_omp
2 make clean    # remove os binarios gerados, se for preciso recompilar do zero
```

Listing 10: Compilação.

O alvo `all` chama o `gcc` com as *flags* `-Wall -Wextra -O2 -fopenmp -std=c99` (Tabela 4); não é necessário passar nenhuma flag manualmente.

```
1 ./fire_seq entrada.txt
2 ./fire_omp entrada.txt
```

Listing 11: Execução, no formato exigido pelo enunciado.

Cada binário recebe apenas o caminho do arquivo de entrada como argumento e escreve as doze linhas de saída (Seção 5) em `stdout`. O número de *threads* T usado por `fire_omp` é lido da própria primeira linha do arquivo de entrada (Seção 2), não de uma variável de ambiente ou de um argumento adicional.